

We will further assume without loss of generality that the hidden state gate is the first $p\Lambda$ gate. The output of the last $p\Lambda$ gate will be the next hidden state. We will also assume that the last $p(\Lambda + d)$ to $p\Lambda - 1$ gates are the output gates. We will now first describe the chain of thought that the Transformer will output and then construct the Transformer.

Chain of thought Taking any input $\mathcal{S}$ with length n , the Transformer will output a sequence of 0 and 1 tokens. The first n tokens will be the same as the input sequence. For each $a \geq 0$ and $b \in [C(n) + 1]$, the $n + a(C(n) + 1) + b$ token is

1. the output of gate b when RNN circuit is calculating the output at a position plus 1, if $b \leq C(n)$.
2. the $n + a + 1$ token in the RNN chain of thought, if $b = C(n) + 1$.

Construction of the Transformer.

1. **Layer 1.** The first attention layer will output zero and the first FFN layer will be of width $O(C(n))$, encoding all the gate information. The output of the first layer at position $n + a(C(n) + 1) + b$ will have the following coordinate:
 - The input i will be encoded in the first dimensions.
 - a, a^2, b, b^2 will be encoded in four different dimensions.
 - The gate type $t(s(b))$ will be encoded in the next dimension, where $s(b) = (b + 1) \bmod (C(n) + 1)$. If $b = C(n) - 1$, then the gate type will be encoded as 0.
 - The necessary dependence $l(s(b)), l^2(s(b))$ and $r(s(b)), r^2(s(b))$ will be encoded in the next two dimensions.
 - A constant 1 will be encoded in the next dimension.
2. **Layer 2 Attention.** Together with the Layer 1 FFN, the Layer 2 Attention will implement two match functions (Definition C.9) to copy the output of gate $l(b + 1)$ and $r(b + 1)$ when RNN circuit is calculating the output at a position. When the type of gate $b + 1$ is not AND or XOR, the result will not be used in the following calculation.
3. **Layer 2 FFN** Layer 2 FFN will be of width $O(1)$. The output of the layer will be
 - When $b < C(n)$ and $t(s(b))$ is AND or XOR or constant, one dimension of the output will be the output of gate $b + 1$ when RNN circuit is calculating the output at a position.
 - When $b < C(n)$ and $t(s(b))$ is an input or hidden state gate or $b = C(n) + 1$, one dimension of the output will be the position in the current chain of thought where the input bit or hidden state bit is copied from and the other dimension will be the square of that position
 - When $b = C(n)$, the output remains the same as the input to Layer 2 FFN.
4. **Layer 3 Attention.** Layer 3 Attention will be of width $O(1)$. Together with Layer 2 FFN, Layer 3 Attention will implement Match heads (Definition C.9) to copy the output at the position where the input bit or hidden state bit is copied from. When the type of gate $b + 1$ is not input or hidden state gate, the result will not be used in the following calculation.
5. **Layer 3 FFN** Layer 3 FFN will be of width $O(1)$. The output of the layer will be
 - When $b \neq C(n)$, one dimension of the output will be output of gate $s(b)$ when RNN circuit is calculating the output at $a + 1[b = C(n) + 1]$ position.
 - When $b = C(n)$, the output remains the same as the input to Layer 3 FFN.
6. **Layer 4** Layer 4 Attention will have $p - 1$ heads and each head will be of width $O(1)$. Head $h \in [p - 1]$ will copy the first dimension of the output of Layer 3 FFN at position $n + a(P(n) + 1) + b - (p - h)$ and weight each of them by $2^{-h+(p-1)/2}$ and add them in one dimension. The Layer 4 FFN will calculate r when the first dimension of the input is 1 and $-r$ otherwise. Hence, for each $a \geq 0$, the $n + a(C(n) + 1) - hp, h \in [\Lambda : \Lambda + d]$ token contains a dimension i_1 which is the $k - \Lambda$ dimension of the output of the RNN at position a .
7. **Layer 5** Layer 5 Attention will have $d + 1$ heads and each head will be of width $O(1)$. Head $h \in [d + 1]$ will copy the dimension i_1 of the output of Layer 4 FFN at position $n + a(P(n) + 1) + b - (h + \Lambda)p$ to a disjoint dimension i_{h+1} . The Layer 5 FFN will then make sure the output of Layer 5 satisfies the following:
 - When $b \neq C(n)$, one dimension of the output will be n times the output of gate $s(b)$ when RNN circuit is calculating the output at $a + 1[b = C(n) + 1]$ position plus 1, which will decode to the corresponding value.

- When $b = C(n)$, the first d dimension of the output will be the same as the output of the RNN at position a , and the rest dimension will be 0, which will decode to the same token as the chain of thought of the RNN at position $a + 1$.

This concludes the construction of the Transformer. □

C.8 Proof of Theorem 5.2

We will first state the theorem for clarity.

Theorem 5.2. *For task $T \in \{\text{Index}, \text{AR}, \text{c-gram retrieval}, \text{Counting}, \text{IsTree}\}$, there exist constant-size Linear RNNs that can solve T with In-context RAG. For T other than IsTree , $O(1)$ steps of CoT is required and for IsTree , $O(n \log n)$ steps of CoT is required.*

Proof. When task T is Index, given an input sequence that ends with a query token k , RNN will generate the following search query sequence: $\langle \text{StartSearch} \rangle, ^{(\text{?} : \backslash S \backslash S^*) \{k-1\} (\backslash S)}, \langle \text{EndSearch} \rangle$.

Then the regular expression will match the k -th token in the input sequence. The RNN needs to recite the format of the query, remember the index k and calculate $k - 1$ to generate the regular expression. As we have shown in Lemmas C.20 and C.23, RNN can recite a fixed sequence at fixed position and memorize the recent input, the above sequence can be generated by an RNN. The explicit construction of the RNN is omitted here.

When task T is AR, given an input sequence that ends with a query token k , RNN will generate the following search query sequence: $\langle \text{StartSearch} \rangle, \backslash b k \backslash b (\backslash S^+) \backslash b, \langle \text{EndSearch} \rangle$.

Then the regular expression will match the next token after the first occurrence of k in the input sequence. The RNN needs to recite the format of the query and remember the query token k to generate the regular expression. The explicit construction of the RNN is omitted here.

When task T is c-gram retrieval, given an input sequence that ends with query tokens, RNN will generate the following search query sequence: $\langle \text{StartSearch} \rangle, \backslash b k_1 \dots k_{c-1} \backslash b (\backslash S^+) \backslash b, \langle \text{EndSearch} \rangle$.

Then the regular expression will match the next token after the first occurrence of $k_1, \dots, k_{c-1}$ in the input sequence. The RNN needs to recite the format of the query and remember the query tokens $k_1, \dots, k_{c-1}$ to generate the regular expression. The explicit construction of the RNN is omitted here.

When task T is Counting, given an input sequence that ends with a query token v and a query threshold k , RNN will generate the following search query sequence $\langle \text{StartSearch} \rangle, (\backslash b v \backslash b) \{k+1\}, \langle \text{EndSearch} \rangle$.

Then the regular expression will match the k -th occurrence of v in the input sequence. The RNN needs to recite the format of the query and remember the query token v and the threshold k to generate the regular expression. The RNN can then check whether the retrieval result is FAILED to determine if the count is less than k . The explicit construction of the RNN is omitted here. □

C.9 Proof of Theorem 5.3

In this section, we will prove Theorem 5.3. We will first state the theorem for convenience.

Theorem 5.3. *There exists a Linear RNN family with $O(\log n)$ bit memory and $O(\log n)$ parameter, that can solve IsTree of size n with In-context RAG in $O(n)$ CoT steps.*

Proof of Theorem 5.3. We will first define the sequence that the retrieval-augmented RNN will generate and then construct an RNN that can generate such a sequence.

Sequence Generation. We will use a variant of Algorithm 1 to generate the sequence and we will use the concatenation of the tokenization of the sequence returned by Algorithm 2 as the sequence that the retrieval augmented RNN will generate.

RNN Construction. We can use similar proof in Theorem 5.4 by having the RNN memorize local sequences and determine the phase of Algorithm 2 it is in. The RNN will maintain the length of S_2 (Lemma C.21) and the top of S_1 in the state (Lemma C.20) and it is easy to check that the retrieval function will retrieve the correct result for each search

Algorithm 2 Depth-First Search Algorithm with Retrieving

Require: A graph $G = (V, E)$ with n vertices and E has an ordering $e_1, \dots, e_m$.

```

1: Initialize two stacks of vertices  $S_1, S_2$  with  $S_1 = [v_1], S_2 = \emptyset$ , a list  $L$  with  $L = \emptyset$ , and a vertex  $v' = \text{FAILED}$ .
2: while  $S_1$  is not empty do
3:   Let  $v$  be the top of  $S_1$ . Push  $v$  to  $L$ .
4:   Generate the regular expression  $r_2 = \backslash b (\backslash S+) \backslash b - \backslash b v \backslash b$ .
5:   Let  $f(v)$  be the predecessor of  $v$  in  $S_1$  for the first time and  $\text{FAILED}$  when  $v = v_1$ .
6:   Push  $\langle \text{StartSearch}, r_2, \langle \text{EndSearch}, f(v) \rangle$  to  $L$ .
7:   if  $v' \neq f(v)$  then
8:     Generate the regular expression
      
$$r_1 = \left( \backslash b v' \backslash b \sim \backslash b v \backslash b \mid \backslash b v \backslash b \sim \backslash b v' \backslash b \right) . * ? \left( \backslash b (\backslash S+) \backslash b \sim \backslash b v \backslash b \mid \backslash b v \backslash b \sim \backslash b (\backslash S+) \backslash b \right)$$

9:   else
10:    Generate the regular expression  $r_1 = \backslash b (\backslash S+) \backslash b \sim \backslash b v \backslash b \mid \backslash b v \backslash b \sim \backslash b (\backslash S+) \backslash b$ 
11:   end if
12:   Push  $\langle \text{StartSearch}, r_1, \langle \text{EndSearch} \rangle$  to  $L$ .
13:   if there exists a neighbor  $u$  of  $v$  such that  $(u, v)$  has larger order than  $(v, v')$  when  $v' \neq f(v)$  or there exists a
      neighbor  $u$  of  $v$  such that  $u \neq f(v)$  when  $v' = f(v)$  then
14:     Choose  $u$  such that edge  $(u, v)$  has the smallest possible order and push  $u$  to  $L$ . Let  $v'' = u$ .
15:   else
16:     Push  $\text{FAILED}$  to  $L$ . Let  $v'' = \text{FAILED}$ .
17:   end if
18:   if  $v'' = f(v) \neq \text{FAILED}$  then
19:     Generate the regular expression
      
$$r_3 = \left( \backslash b v'' \backslash b \sim \backslash b v \backslash b \mid \backslash b v \backslash b \sim \backslash b v'' \backslash b \right) . * ? \left( \backslash b (\backslash S+) \backslash b \sim \backslash b v \backslash b \mid \backslash b v \backslash b \sim \backslash b (\backslash S+) \backslash b \right)$$

20:     Push  $\langle \text{StartSearch}, r_3, \langle \text{EndSearch} \rangle$  to  $L$ .
21:     if there exists a neighbor  $u$  of  $v$  such that  $(u, v)$  has larger order than  $(v, v'')$  then
22:       Choose  $u$  such that edge  $(u, v)$  has the smallest possible order and push  $u$  to  $L$ . Let  $v'' = u$ .
23:     else
24:       Push  $\text{FAILED}$  to  $L$ . Let  $v'' = \text{FAILED}$ .
25:     end if
26:   end if
27:   if  $v'' = \text{FAILED}$  then
28:     Pop  $v$  from  $S_1$ . Push  $v$  to  $S_2$ . Let  $v' = v$ .
29:   else
30:     Generate the regular expression  $r_4 = \backslash b (\backslash S+) \backslash b - \backslash b v'' \backslash b$ 
31:     Push  $\langle \text{StartSearch}, r_4, \langle \text{EndSearch}, 0 \rangle$  to  $L$ .
32:     if  $v''$  is not in  $S_1$  then
33:       Push  $\text{FAILED}, v, -, v''$  to  $L$ .
34:       Push  $v''$  to  $S_1$ . Let  $v' = v$ .
35:     else
36:       Let  $f(v'')$  be the predecessor of  $v$  in  $S_1$  for the first time and  $\text{FAILED}$  when  $v'' = v_1$ .
37:       Push  $f(v'')$ ,  $\text{NO}$  to  $L$ .
38:       return  $L$ .
39:     end if
40:   end if
41: end while
42: if  $S_2$  has  $n$  vertices then
43:   Push  $\text{YES}$  to  $L$ .
44:   return  $L$ .
45: else
46:   Push  $\text{NO}$  to  $L$ .
47:   return  $L$ .
48: end if

```